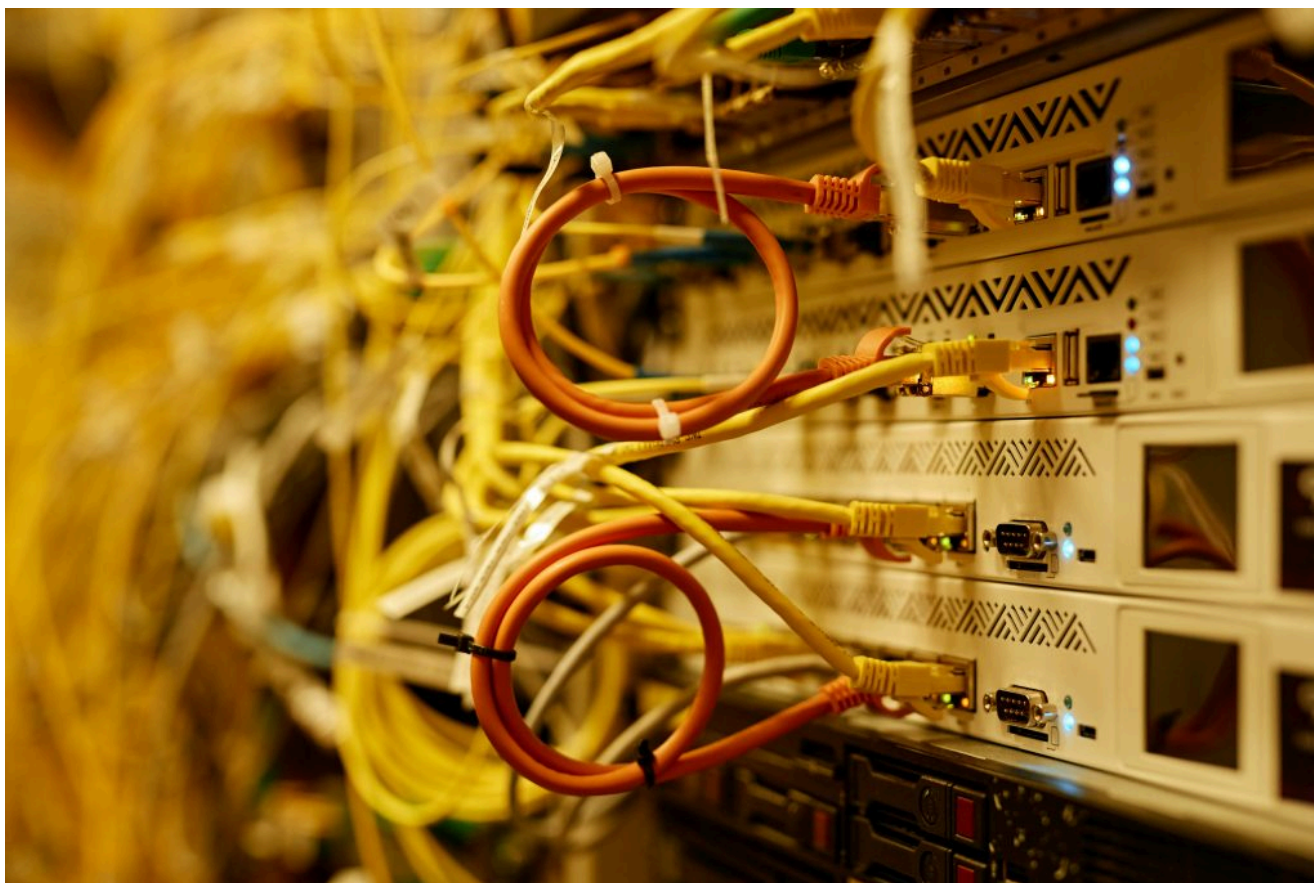


УПРАВЛЕНИЕ API / ИНСТРУМЕНТЫ ДЛЯ РАЗРАБОТЧИКОВ / ПРОТОКОЛ MODEL CONTEXT

Самое масштабное обновление MCP устраняет механизм, на котором были построены многие серверы

В самой масштабной версии MCP были удалены сеансы и процесс инициализации, состояние перенесено в дескрипторы, а удаленные серверы стали работать по принципу HTTP без сохранения состояния.

26 июля 2026 года, 12:00 [Джанакiram MSV](#)



Getty Images для Unsplash+

Самое масштабное обновление Model Context Protocol (MCP) с момента его запуска уже не за горами. Ведущие разработчики **заморозили** релиз-кандидат 21 мая, а окончательная спецификация должна выйти 28 июля.

Первый взгляд, **список изменений** выглядит радикальным: сеансы и процесс инициализации будут упразднены, а три основные функции

привычных задач в инфраструктуру, уже созданную для их выполнения. Для операторов это решение давней проблемы: для запуска удаленного сервера MCP не требуется специализированное оборудование, которое не нужно для обычного сервиса без сохранения состояния.

Налог, который MCP взимала со своих пользователей

Чтобы понять, почему MCP была удалена, нужно изучить разницу между изначальным замыслом MCP и ее фактическим применением. Самая ранняя и наиболее узнаваемая форма MCP представляла собой настольное приложение, взаимодействующее с локальным процессом через стандартный ввод и вывод, где рукопожатие при запуске было недорогим и выполнялось по долгосрочному соединению.

Команда, разрабатывающая сервер MCP, заплатила за решение проблемы распределенных систем, которую создал протокол.

По мере того как все больше серверов переходило на удаленные горизонтально масштабируемые развертывания, эти сеансы стали проблемой. Сервер создавал `Mcp-Session-Id`, который привязывал клиента к экземпляру, выдавшему его. Для горизонтального масштабирования часто требуется привязка к сеансу, внешнее хранилище сессий или логика шлюза, поддерживающая MCP и анализирующая тела JSON для определения маршрутизации вызовов. Команда, разрабатывающая сервер MCP, платила за решение проблемы распределенных систем, которую создал для них протокол.

Согласование возможностей требовало дополнительных затрат. Поскольку обмен возможностями происходил один раз при подключении, результаты поиска могли различаться в зависимости от подключения, что затрудняло кэширование между сеансами или в общем промежуточном хранилище.

Что оптимизировали разработчики

Шесть предложений по усовершенствованию спецификации направлены на достижение одной цели: сделать так, чтобы каждый запрос существовал сам себе. Версия протокола и возможности клиента теперь передаются в

позволяет запрашивать возможности сервера независимо друг от друга — либо заранее, либо по мере необходимости. В сообщении о выпуске это названо основополагающим принципом «плати по мере использования»: ядро остается компактным, а сохранение состояния используется только там, где это действительно необходимо.

The release post calls the underlying principle pay-as-you-go complexity, where the core stays lean, and statefulness shows up only where a feature genuinely needs it.

The obvious objection is that plenty of servers do need to remember things. One principal answer is the explicit handle, which is the pattern every shopping cart built over HTTP has used for twenty years. A tool mints a `basket_id`, returns it in the result, and the model passes it back as an ordinary argument on the next call. Account state, resource URIs, task handles, and ordinary database identifiers all remain available for anything that should not travel that route.

TRENDING STORIES

1. MCP's biggest update removes the machinery many servers were built around
2. The case for running AI agents on Markdown files instead of MCP servers
3. MCP's biggest growing pains for production use will soon be solved
4. OpenAI's Codex gets plugins
5. Moving AI apps from prototype to production requires enterprise-grade postgres infrastructure

What makes the handle more than a workaround is that it is visible to the model. Session state hidden in transport metadata was something the model could never reason about, whereas a handle in a tool result can be composed across tools and ded between workflow steps. The trade-off is that it also appears in prompts,

What developers actually get

For server authors, a remote MCP server can now be operated much more like a conventional stateless HTTP service. Three replicas behind round-robin, no affinity configuration, and no protocol session store to run or recover. Rolling deploys no longer invalidate sessions or strand a client on a removed instance, though they can still interrupt in-flight requests and subscription streams, and since resumability is gone, the client reissues that work under a new request ID.

For platform teams, the required `Mcp-Method` header, plus `Mcp-Name` for the named tool, resource, and prompt operations, means a gateway can rate-limit or authorize by operation without inspecting a request body. That only holds under the **transport validation rules**, where the backend rejects any header that disagrees with the body and a policy-enforcing intermediary refuses protocol versions that do not guarantee the check. Skip that and a benign header can front a different call underneath.

The caching change deserves more attention than it has received. Affected list and read results must now include `tTlMs` and `cacheScope`, modeled on HTTP `Cache-Control`, so a client can hold a catalog for a stated interval rather than refetching it. The draft treats `tTlMs` as a freshness hint rather than a promise that the data still holds. Servers are also asked to return tools in deterministic order, which the draft ties to better prompt cache hit rates, and at volume that can mean lower latency or token cost wherever the provider prices prompt caching.

One caveat belongs here rather than in a footnote. Statelessness at the protocol layer buys routability, not determinism. Two replicas will accept the same request without consulting protocol state, and they will still return different answers if they run different versions or read different downstream data.

Why care about extensions

The ecosystem argument is stronger than any single feature. **Extensions** now get namespaced identifiers, official ones under `io.modelcontextprotocol` and third-party ones under a reversed domain the author owns, plus their own `ext-` repositories and release cadences. Anyone who has written a Kubernetes custom resource definition will recognize the shape, where a capability ships and matures side the core release train.

redesigned as an extension. Moving it out of core is itself a breaking specification change, but the next iteration is not, because extensions evolve through capability flags or settings-level versioning, with a fresh identifier only when a break is unavoidable. MCP Apps, already available as an extension, now sits inside a formally governed negotiation framework rather than a process that did not exist.

What the deprecation policy buys

The **feature lifecycle policy** gives every feature an Active, Deprecated, and Removed state, with a minimum twelve-month window measured from the revision in which the feature first becomes Deprecated. The floor can be shortened only for an active security risk carrying a published advisory or documented exploitation, and even then ninety days is the hard minimum. A public registry lists what is on the way out and by when, and a Standards Track proposal cannot reach Final until a matching scenario lands in the conformance suite.

For a developer, that reads like housekeeping. For anyone who has to justify an MCP integration to a platform review board, a written deprecation guarantee is worth more than any capability in the release.

What it costs

None of this arrives free of charge. Anyone who built against the experimental Tasks API migrates to the new lifecycle, and any server that issued its own requests to the client moves onto the **Multi Round-Trip Requests** pattern, where the server returns what it still needs, and the client retries with the answers. That server also has to authenticate any echoed `requestState` that can influence authorization or business logic, and one-time workflows still need their own replay tracking.

The deprecations are migration directions rather than drop-in replacements, and Sampling is the sharpest case. A server using client-mediated Sampling needed no provider credentials and generally did not carry the model bill, whereas calling a provider API directly turns it into a credential holder, a billing party, and a separate processor of user data. Logging tells a similar story, since `stderr` and OpenTelemetry answer operator observability without giving a remote client any equivalent of the structured log stream it used to receive.

state going away.

Application state does not disappear either. Handles, carts, task records, and idempotency keys still need somewhere to live. The protocol stopped managing state, which is not the same as the state going away.

The road ahead

Removing a foundational abstraction less than two years into a protocol's life is a calculated risk, and the maintainers hedged it sensibly with a ten-week validation window and beta SDKs for Python, TypeScript, Go, and C#.

They also defined a migration path where clients probe with `server/discover` first and fall back to `initialize` — only when they encounter a legacy-only server. It's a wire-level break with a negotiated path across it rather than an ecosystem flag day.

The release-candidate window is the right time to inventory session dependencies and test them, with production rollout following ratification and stable SDKs. From individual server authors to platform teams to the vendors building gateways and registries around it, the payoff is a protocol layer that fits far more naturally onto infrastructure the industry already knows how to operate.

TNS



Janakiram MSV (Jani) is a practicing architect, research analyst, and advisor to Silicon Valley startups. He focuses on the convergence of modern infrastructure powered by cloud-native technology and machine intelligence driven by generative AI. Before becoming an entrepreneur, he spent...

[Read more from Janakiram MSV →](#)

SHARE THIS STORY



-
2. The case for running AI agents on Markdown files instead of MCP servers
 3. MCP's biggest growing pains for production use will soon be solved
 4. OpenAI's Codex gets plugins
 5. Moving AI apps from prototype to production requires enterprise-grade postgres infrastructure

INSIGHTS FROM OUR SPONSORS

Four Meko Questions, Answered

4 August 2026

How to Persist AI Agent Context Deterministically

31 July 2026

“Where Were We?” Watch Meko Give AI Agents a Shared Context

16 July 2026



A 13-year-old PostgreSQL bug, found by running Postgres on Kubernetes with CloudNativePG

3 August 2026

Beyond the DBaaS Trap: Claiming Operational Independence for PostgreSQL

26 February 2026

Why the Future of Postgres Extensions in Kubernetes is Immutable and Flexible

18 March 2025



When Column Names Lie: Bringing External Catalogs to NL2SQL++

3 August 2026

What We Learned Evaluating Agent Memory: The Results (Part 2)

29 July 2026

What We Learned Evaluating Agent Memory: The Setup (Part 1)

28 July 2026

Receive a free roundup of the most recent TNS articles in your inbox each day.

svo1975pvn1982@gmail.com

SUBSCRIBE

The New Stack does not sell your information or share it with unaffiliated third parties. By continuing, you agree to our [Terms of Use](#) and [Privacy Policy](#).

ARCHITECTURE

Cloud Native Ecosystem
Containers
Databases
Edge Computing
Infrastructure as Code
Linux
Microservices
Open Source
Networking
Storage

OPERATIONS

DevOps
CI/CD

ENGINEERING

AI
AI Engineering
API Management
Backend development
Data
Frontend Development
Large Language Models
Security
Software Development
WebAssembly

CHANNELS

Podcasts
Ebooks

Kubernetes

Observability

Operations

Platform Engineering

Newsletter

TNS RSS Feeds

THE NEW STACK

About / Contact

Sponsors

Advertise With Us

Contributions

 roadmap.sh

Community created roadmaps, articles, resources and journeys for developers to help you choose your path and grow in your career.

Frontend Developer Roadmap

Backend Developer Roadmap

Devops Roadmap

FOLLOW TNS



© The New Stack 2026

Disclosures

Terms of Use

Advertising Terms & Conditions

Privacy Policy

Cookie Policy